

SGLang HiSparse 容量模型

HBM、并发与 Host Memory 的容量推导

SGLang HiSparse 将 KV / Indexer cache 分为 logical 命名空间、GPU device hot Buffer 与 CPU host cold tier。本文针对 SGLang 的 **ratio-driven 分配策略** 建立容量模型，用来判断固定 HBM 预算下的 worker 容量、可达并发、Buffer / Indexer 显存构成以及 host 内存需求。

1. 背景与理论假设

本文采用两个基础假设：

1. **Indexer cache 全量常驻 GPU。** HiSparse 只在 host 与 device 之间分层 MLA KV；Indexer 覆盖完整 logical token 空间。层共享系数为 $R_{Share} = N_{layer} / N_{layer, indexer}$ 。
2. **暂不考虑 Prefix Cache。** 每个 logical token 独立占用 MLA KV / Indexer 空间。带缓存模型留到第 8 节继续扩展。

2. 符号定义

符号	含义	典型值
$N_{T, Buffer}$	静态均分时每条请求的 device hot slots	6144
B_{Buffer}^T	worker / batch 的总 device hot slots	$N_{bs, max} N_{T, Buffer}$
R_{Host}	SGLang logical-capacity expansion factor: 每个 GPU MLA hot slot 对应的 logical Indexer token 数, 即 C_{Batch} / B_{Buffer}^T	hisparse_host_to_device_ratio, 且 $R_{Host} \geq 1$
$C = C_{Batch}$	一个 decode batch 能服务的 aggregate logical tokens	由 R_{Host} 定义并在引理 1 中推导其 HBM 上界

符号	含义	典型值
N_{Topk}	sparse kernel 的 per-request Buffer 下界	2048
R_{Share}	Indexer 层共享系数	GLM5.1: 1; GLM5.2: 3.714
$B_{T,Indexer}$	每 logical token 每 Indexer 层字节数	132 B
$B_{T,MLA}$	每 hot token 每层 MLA KV 字节数	656 B
N_{layer}	模型层数	78
$L_{max,model}$	单请求模型上下文上限	256K / 1M
M_{cache}	可用于 HiSparse Buffer 与 Indexer 的 HBM	$M_{Available}^F - M_{weights}$
$M_{Host,Available}$	worker 可用于 cold MLA KV 的 Host 内存	部署参数

3. Capacity 推导

3.1 定义 host/device 缩放系数

SGLang 将 R_{Host} 定义为 full logical Indexer namespace 相对于 GPU MLA hot Buffer 的 **logical-capacity expansion factor**:

$$R_{Host} := \frac{C_{Batch}}{B_{Buffer}^T} = \frac{size_{full}}{size_{device}}, R_{Host} \geq 1, \quad (3.1)$$

$$C = C_{Batch} = R_{Host} B_{Buffer}^T.$$

这是 token-slot 数量之比，不是 Host RAM 与 HBM 的字节数之比；它是 SGLang allocator 的输入参数，而不是分配完成后测得的派生量。对应到 SGLang allocator，即 $size_{full} = size_{device} \cdot R_{Host}$ 。因此，式 (3.1) 是 logical 命名空间的定义，不是完成显存分配后才得到的派生关系。后续推导要解决的是：在 HBM 预算约束下， B_{Buffer}^T 最大能有多大，以及由式 (3.1) 决定的 C_{Batch} 最大能有多大。

3.2 拆分 HBM 预算

MLA device Buffer 只保存 hot token slots，而全量常驻 GPU 的 Indexer 需要覆盖全部 C_{Batch} 个 logical tokens。两部分 HBM 占用分别为：

$$\begin{aligned} M_{Buffer} &= N_{layer} B_{Buffer}^T B_{T,MLA}, \\ M_{Indexer} &= N_{layer} \frac{C_{Batch} B_{T,Indexer}}{R_{Share}}. \end{aligned} \quad (3.2)$$

因此，SGLang 最基础的 HBM 预算式为：

$$M_{cache} \geq N_{layer} B_{Buffer}^T B_{T,MLA} + N_{layer} \frac{C_{Batch} B_{T,Indexer}}{R_{Share}} \quad (3.3)$$

将式 (3.1) 的缩放定义代入：

$$M_{cache} \geq B_{Buffer}^T N_{layer} \left(B_{T,MLA} + \frac{R_{Host} B_{T,Indexer}}{R_{Share}} \right). \quad (3.4)$$

由此也可以得到一个 hot slot 的等效 HBM 成本：

$$m_{slot} = N_{layer} \left(B_{T,MLA} + \frac{R_{Host} B_{T,Indexer}}{R_{Share}} \right) \quad (3.5)$$

3.3 反解 worker-wide 总 Buffer

因此，充分使用 HBM 时，worker-wide 总 Buffer 为：

$$B_{Buffer}^T = \frac{M_{cache}}{m_{slot}} = \frac{M_{cache}}{N_{layer} \left(B_{T,MLA} + \frac{R_{Host} B_{T,Indexer}}{R_{Share}} \right)} \quad (3.6)$$

未充分使用 HBM 时，式 (3.6) 的等号相应变为小于号。本文图片采用充分使用 HBM 的容量上界。

3.4 Batch Capacity

引理 1 (HiSparse Batch Capacity) . 在前述理论假设和充分使用 HBM 的条件下，一个 Decode batch 能服务的 aggregate logical token 数为：

$$\begin{aligned}
 C &= C_{Batch} = R_{Host} B_{Buffer}^T \\
 &= \frac{M_{cache} R_{Host}}{N_{layer} \left(B_{T,MLA} + \frac{R_{Host} B_{T,Indexer}}{R_{Share}} \right)} \\
 &= \frac{M_{cache} R_{Host}}{m_{slot}}
 \end{aligned} \tag{3.7}$$

证明。 式 (3.1) 首先定义 worker-wide Buffer 所代表的 logical capacity。将该定义代入式 (3.3) 的两项 HBM 预算，得到式 (3.4)；令 HBM 预算充分使用后得到式 (3.6) 的 Buffer 上界；最后将该上界代回式 (3.1)，即可得到式 (3.7)。

式 (3.7) 是本文图片中 Capacity 曲线使用的核心公式。

3.5 与 batch size 的关系

静态均分只是 worker-wide Buffer pool 的一种切分方式：

$$B_{Buffer}^T = N_{bs,max} N_{T,Buffer} \tag{3.8}$$

将该关系代入式 (3.1) 的缩放定义：

$$C_{Batch} = N_{bs,max} R_{Host} N_{T,Buffer} = N_{bs,max} C_{host,seq} \tag{3.9}$$

其中 $C_{host,seq} = R_{Host} N_{T,Buffer}$ 。式 (3.8)-(3.9) 只描述总 Buffer 如何静态切分；固定 M_{cache} 和 R_{Host} 时，引理 1 中的 worker-wide Capacity 不随 $N_{bs,max}$ 改变。

4. Capacity 约束

Capacity 模型依次应用以下三个约束。

约束 1 (Sparse 与 GPU-HBM 可行性) . Per-sequence Buffer 必须同时满足 sparse kernel 的 Top-k 下界和 GPU-HBM 上界。

Sparse kernel 首先给出硬下界：

$$N_{T,Buffer} \geq N_{Topk} \quad (4.1)$$

令 $M_{cache} = M_{Available} F - M_{weights}$, HBM 则给出静态均分下每条请求的 Buffer 上界：

$$N_{T,Buffer} \leq \frac{M_{cache}}{N_{layer} N_{bs,max} \left(B_{T,MLA} + \frac{R_{Host} B_{T,Indexer}}{R_{Share}} \right)} \quad (4.2)$$

worker-wide 总 Buffer 上界为：

$$B_{Buffer}^{T,GPU} = \frac{M_{cache}}{N_{layer} \left(B_{T,MLA} + \frac{R_{Host} B_{T,Indexer}}{R_{Share}} \right)} \quad (4.3)$$

它与 $N_{bs,max}$ 无关。

合并式 (4.1) 与式 (4.2), 即可得到 per-sequence Buffer 的 sparse 可行区间。式 (4.3) 给出与之等价的 worker-wide 总 Buffer 上界, 并说明它与 $N_{bs,max}$ 无关。

约束 2 (Model-length Coverage) . Worker-wide Capacity 至少应覆盖一个 model-length logical context。

本文图片使用一个 model-length 的 worker aggregate capacity 作为目标：

$$C_{Batch} \geq L_{max,model} \quad (4.4)$$

最多 $N_{bs,max}$ 条请求能访问的 aggregate logical token 不超过：

$$C_{Batch} \leq N_{bs,max} L_{max,model} \quad (4.5)$$

这是避免浪费的上界, 不是 GPU 可行性的硬约束。

约束 3 (Host 物理内存上界) . Host cold tier 必须为 worker 覆盖的所有 logical token 提供物理 MLA KV 空间。

Host cold tier 必须为 logical token space 提供 MLA KV:

$$C_{Batch} N_{layer} B_{T,MLA} \leq M_{Host,Available} \quad (4.6)$$

因此:

$$C_{Batch} \leq C_{Batch,max}^{Host} = \frac{M_{Host,Available}}{N_{layer} B_{T,MLA}} \quad (4.7)$$

5. HBM 到底花在哪里

5.1 Buffer 与 Indexer 构成

Buffer / Indexer 的两项显存构成已由式 (3.2) 定义, 并由式 (3.3) 合并到同一个 HBM 预算中: MLA Buffer 只覆盖 B_{Buffer}^T 个 GPU hot slots, 而 Indexer 覆盖完整的 C_{Batch} logical namespace。图片中的 Memory Composition 正是对这两项的可视化, 不是另一套容量模型。

在“Indexer 全量常驻 GPU”的假设下, host cold tier 只保存 MLA KV, 不重复保存 Indexer。

5.2 Indexer 占比

$$\frac{M_{Indexer}}{M_{Buffer}} = \frac{R_{Host} B_{T,Indexer}}{R_{Share} B_{T,MLA}} \quad (5.1)$$

Indexer HBM 占比为:

$$f_{Indexer} = \frac{R_{Host} B_{T,Indexer}}{R_{Share} B_{T,MLA} + R_{Host} B_{T,Indexer}} \quad (5.2)$$

Buffer 与 Indexer 各占 50% 时:

$$R_{Host}^{50pct} = \frac{R_{Share} B_{T,MLA}}{B_{T,Indexer}} \quad (5.3)$$

GLM5.1 的分界点约为 4.97，GLM5.2 约为 18.46。

5.3 并发与总 Buffer

在固定 HBM 预算、充分使用 Buffer pool 时：

$$N_{T,Buffer}^{GPU} \propto \frac{1}{N_{bs,max}} \quad (5.4)$$

但 $B_{Buffer}^{T,GPU} = N_{bs,max} N_{T,Buffer}^{GPU}$ 保持不变。更高并发不会创造更多 hot slots，只会把同一个 pool 切得更细。

5.4 Host ratio 的 Indexer 天花板

$$C_{Batch}^{GPU}(R_{Host}) = \frac{M_{cache} R_{Host}}{N_{layer} \left(B_{T,MLA} + \frac{R_{Host} B_{T,Indexer}}{R_{Share}} \right)} \quad (5.5)$$

当 $R_{Host} \rightarrow \infty$ ：

$$\lim C_{Batch}^{GPU} = \frac{M_{cache} R_{Share}}{N_{layer} B_{T,Indexer}} \quad (5.6)$$

5.5 最大 batch size 限制 ratio 可达性

每条活跃请求至少需要 N_{Topk} ，因此：

$$N_{bs,max} \leq \frac{M_{cache}}{N_{layer} N_{Topk} \left(B_{T,MLA} + \frac{R_{Host} B_{T,Indexer}}{R_{Share}} \right)} \quad (5.7)$$

固定 $N_{bs,max}$ 后得到：

$$R_{Host,max}^{Topk} = \frac{R_{Share}}{B_{T,Indexer}} \left(\frac{M_{cache}}{N_{layer} N_{bs,max} N_{Topk}} - B_{T,MLA} \right) \quad (5.8)$$

长度均为 L_{req} 时，可接纳请求数满足：

$$N_{req}^{admit} \leq \min \left(N_{bs,max}, \left\lfloor \frac{C_{Batch}^{GPU}}{L_{req}} \right\rfloor, \left\lfloor \frac{B_{Buffer}^{T,GPU}}{N_{Topk}} \right\rfloor \right) \quad (5.9)$$

固定 per-sequence Buffer 为 B 时，图中的连续请求数上界是 $B_{Buffer}^{T,GPU} / B$ 。因此随着 R_{Host} 增大，aggregate capacity 上升，但固定 Buffer 下的最大请求数下降。

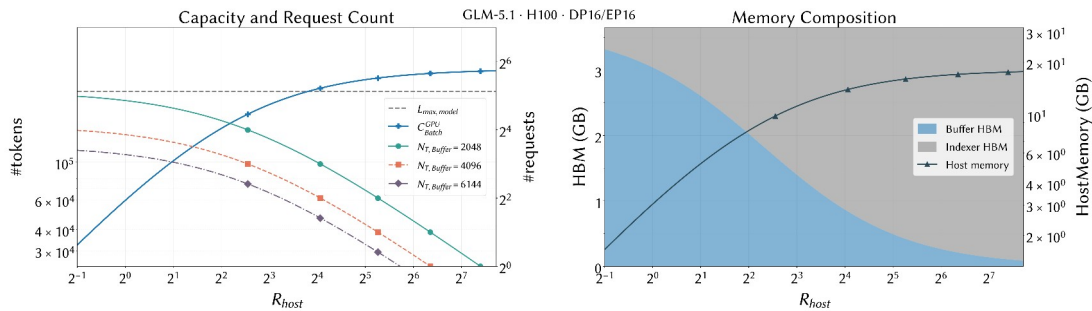
6. GLM5.1 / GLM5.2 实例

配置	$M_{Available}$	F	$M_{weights}$	$M_{cache} = M_{Available} F - M_{weights}$
H100 DP16EP16	77.47 GB	0.88	64.52 GB	3.6536 GB
H100 DP32EP32	77.47 GB	0.82	43.42 GB	20.1054 GB

每张图现在包含两个子图：左图为 C_{Batch}^{GPU} 与固定 Buffer 为 2048 / 4096 / 6144 时的最大请求数；右图为 Buffer / Indexer HBM 的绝对 GB 构成及对应 HostMemory。横轴均从 2^{-1} 开始使用 base-2 对数刻度。

图中的 marker 放在各 N_{bs} 理论 top-k ratio 边界上，而不是任意采样点。由于 $2048 = N_{Topk}$ ，Buffer=2048 曲线上的 marker 会精确落在对应的 N_{bs} 值；Buffer=4096/6144 则分别落在 $N_{bs}/2$ 与 $N_{bs}/3$ 。

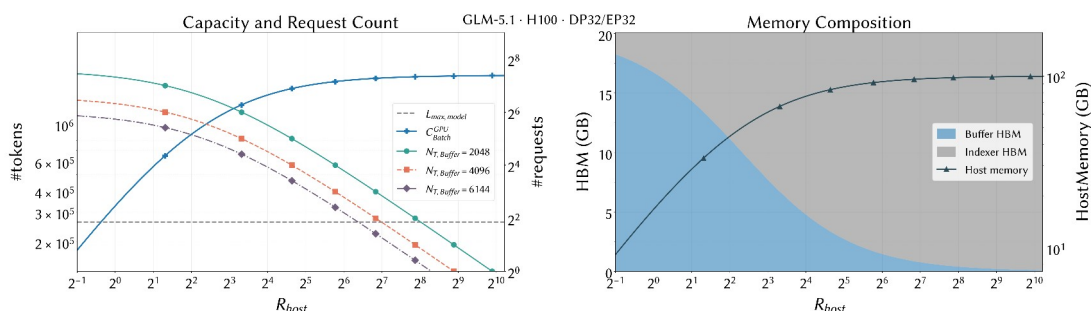
6.1 GLM5.1 (256K)



GLM5.1 H100 DP16EP16 worker capacity

图结论：增大 R_{Host} 会提高 worker aggregate capacity, 但会降低固定 per-sequence Buffer 下的最大请求数；更大的 Buffer 用更低并发换取更大的 hot window。

DP16EP16 的 256K lower crossing 为 $R_{Host} \approx 14.05$ 。 $N_{bs,max} = 1/2/4/8$ 的可行上界分别约为 128/81.67/38.35/16.69。

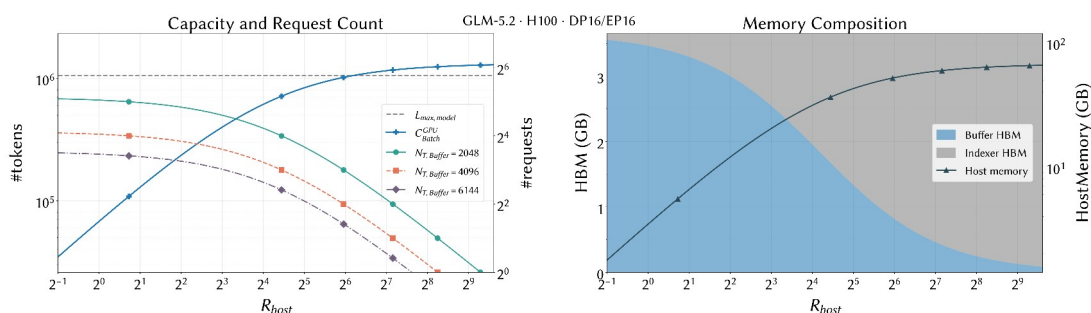


GLM5.1 H100 DP32EP32 worker capacity

图结论：更大的 M_{cache} 同时降低容量 crossing, 并把各并发的 top-k ratio ceiling 推向右侧。

DP32EP32 的 256K lower crossing 为 $R_{Host} \approx 0.77$ ； $N_{bs,max} = 1/2/4/8$ 的上界约为 128/256/233.4/114.2。

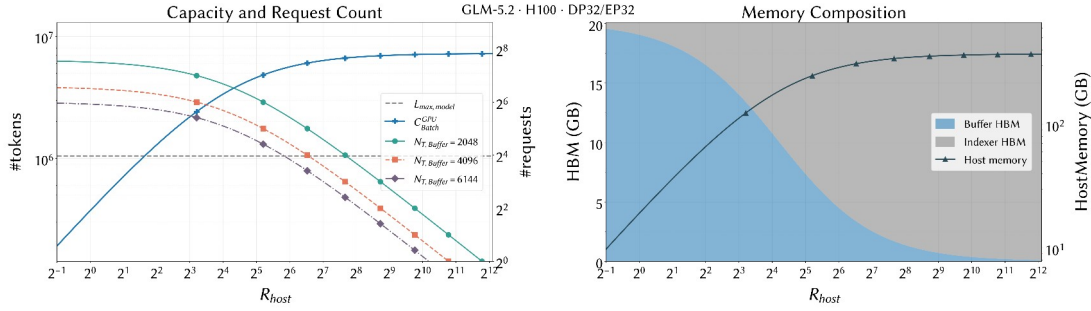
6.2 GLM5.2 (1M)



GLM5.2 H100 DP16EP16 worker capacity

图结论：1M target 把 lower crossing 推到 $R_{Host} \approx 71.85$ ； $N_{bs,max} = 8$ 的 top-k ceiling 位于 crossing 左侧，因此不可达。

$N_{bs,max} = 1/2/4$ 的可行上界约为 512/303.3/142.4。



GLM5.2 H100 DP32EP32 worker capacity

图结论： DP32EP32 将 1M crossing 降到 $R_{Host} \approx 3.12$ ，并使 $N_{bs,max}=1/2/4/8$ 可达。

对应上界约为 512/1024/866.9/424.2。

7. 算例： GLM5.2 + H100 DP16EP16

取 $N_{bs,max}=8$ 、 $R_{Host}=2$ 、 $R_{Share}=3.714$ 、 $N_{Topk}=2048$ 、 $N_{T,Buffer}=6144$ ：

1. $6144 \geq 2048$ ，满足 sparse floor。
2. GPU per-sequence Buffer 上界约为 8053，因此默认 Buffer 可运行。
3. $C_{Batch} = 8 \times 2 \times 6144 = 98,304$ ，低于 1M target。容量 crossing 位于 71.85，但 $N_{bs,max}=8$ 的 top-k ceiling 约为 62，两者没有交集；降到 $N_{bs,max}=4$ 后得到 $[71.85, 142.4]$ 。

8. 调参指南

目标	建议
sparse kernel 可运行	$N_{T,Buffer} \geq N_{Topk}$
一个 model-length 的 worker aggregate capacity	令 $C_{Batch} \geq L_{max,model}$ ，并确认 crossing 小于 $R_{Host,max}^{Topk}$
高并发	使用 worker-wide Buffer pool，并验证 $B_{Buffer}^{T,GPU} \geq N_{bs,max} N_{Topk}$
更高容量	增大 R_{Host} ，但注意 Indexer ceiling 与 host RAM
Indexer 已主导 HBM	优先增大 R_{Share} 或压缩 Indexer，而不是继续推高 R_{Host}